ICC-362 INTRODUCCION A LA INTELIGENCIA ARTIFICIAL

Lab #1 – Agentes de Búsqueda en Pacman

Fecha de Entrega: 19/2/2024

En este proyecto, te familiarizarás con Pac-Man World. En esta asignación, el agente Pac-Man encontrará caminos a través de su mundo laberíntico, tanto para llegar a un lugar concreto como para recoger alimentos de manera eficiente. También creará algoritmos de búsqueda generales y los aplicará a escenarios de Pac-Man.

El código base de este proyecto consta de varios archivos Python, algunos de los cuales necesitarás leer y entender para completar las tareas, y algunas de las cuales puedes ignorar. El código Pac-Man fue desarrollado por John DeNero y Dan Klein en UC Berkeley para la clase CS188.

Importante: Antes de comenzar, asegúrese de tener Python 3.X disponible en la computadora/cuenta en la que estará usando. Puedes consultar la versión de Python ingresando el comando:

```
$ python3 –versión
Python 3.12.3
```

Instalación de los Archivos

El código para este proyecto consiste en varios archivos Python, algunos de los cuales tendrás que leer y entender para completar la asignación, y algunos de los cuales puedes ignorar. Puede descargar todo el código y los archivos de soporte del archivo search.zip. que se adjunta a la asignación.

Un resumen de los archivos se da a continuación:

Archivos que editarás:

search.py	Donde todos tus algoritmos de búsqueda residirán.
searchAgents.py	Donde todos sus agentes basados en registros residirán.

Archivos que tal vez quieras ver:

pacman.py	El archivo principal que ejecuta juegos de Pacman. Este archivo describe un tipo Pacman GameState, que utiliza en este proyecto.
game.py	La lógica detrás de cómo funciona el mundo de Pacman. Este archivo describe varios tipos de soporte como AgentState, Agent, Direction y Grid.
util.py	Estructuras de datos útiles para implementar algoritmos de búsqueda.

Archivos de soporte que puede ignorar:

pacman.py	El archivo principal que ejecuta juegos de Pacman. Este archivo describe un tipo Pacman GameState, que utiliza en este proyecto.
graphicsDisplay.py	Gráficos para Pacman.
graphicsUtils.py	Soporte para gráficos de Pacman.

textDisplay.py	Gráficos ASCII para Pacman
ghostAgents.py	Agentes para controlar fantasmas
keyboardAgents.py	Interfaz de teclado para controlar Pacman
layout.py	Código para leer los archivos de diseño de laberitos y almacenar su contenido.
autograder.py	Auto-corrector del proyecto.
testParser.py	Analizador de archivos de prueba y solución del auto-corrector.
testClasses.py	Clases de prueba general del auto-corrector.
test_cases/	Directorio que contiene los casos de prueba para cada pregunta.
searchTestClasses.py	Las clases específicas de prueba de auto-corrector del proyecto 1

Archivos para Editar y Enviar: Como parte de esta asignación, usted solo editara porciones de search.py y searchAgents.py durante la asignación. Una vez que haya completado la asignación, usted presentará estos archivos a la PVA. Por favor no cambie los otros archivos de esta distribución.

Evaluación: Su código será corregido de forma automática por un auto-corrector para la corrección del código. Por favor, no cambie los nombres de ninguna función o clases proporcionadas dentro del código, o causará estragos en el auto-corrector. Sin embargo, la corrección de su implementación por el profesor, no los resultados de su auto-corrector local, será el juez final de su puntuación. Si es necesario, revisaremos y clasificaremos las asignaciones de forma individual para asegurarnos de que usted reciba el debido crédito por su trabajo.

Bienvenida a Pacman y Comprobación de la Instalación

Después de descargar el código, descomprimiéndolo y cambiando al directorio, usted debe ser capaz de jugar un juego de Pacman usando el teclado, escribiendo lo siguiente en la línea de comandos:

```
$ python3 pacman.py
```

Pacman vive en un mundo de retorcidos pasillos, deliciosas golosinas redondas y fantasmas que tratan de eliminarlo. Navegar a este mundo de manera eficiente, sin fantasmas, será el primer paso de Pacman en dominar su dominio.

El agente más simple en searchAgents.py se llama el GoWestAgent, que siempre se mueve al oeste (un agente reflejo trivial). Este agente puede ganar de vez en cuando bajo circunstancias favorables. Escriba el comando de más abajo para probar este agente en una situación ventajosa:

```
$ python3 pacman.py --layout testMaze --pacman GoWestAgent
```

Pero, las cosas se ven feas para este agente cuando se requiere que gire cuando:

```
$ python3 pacman.py --layout tinyMaze --pacman GoWestAgent
```

Si Pacman se queda atascado, puedes salir del juego escribiendo CTRL-C en tu terminal o cerrar la ventana con el ratón.

Pronto, su agente no sólo resolverá tinyMaze, pero cualquier laberinto que quieras.

Tenga en cuenta que pacman.py apoya una serie de opciones que pueden expresarse con un nombre verboso (por ejemplo, --layout) o un corto camino (por ejemplo, -l). Puedes ver la lista de todas las opciones y sus valores predeterminados a través de:

```
$ python pacman.py -h
```

Nueva sintaxis

Es posible que no haya visto esta sintaxis antes:

```
def my_function(a: int, b: Tuple[int, int], c: List[List], d: Any, e: float=1.0):
```

Esta sintaxis está anotando el tipo de los argumentos que Python debe esperar para esta función. En el ejemplo siguiente, a debería ser un entero, b debería ser una tupla de 2 enteros, c debería ser una lista de listas de cualquier tipo, por lo tanto, una matriz 2D de cualquier tipo, d es esencialmente lo mismo que no haber anotado nada y puede por cualquier tipo, y e debería ser un numero flotante con un valor por defecto de 1.0 si no se pasa en esa posición de los argumentos. Es decir:

```
my_function(1, (2, 3), [['a', 'b'], [None, my_class], [[]]], ('h', 1))
```

La llamada anterior se ajusta al tipo de anotaciones, y no pasa nada en lugar de e, dando por defecto un valor de 1.0 a este parámetro.

Las anotaciones de tipo están destinadas a ser una adición a las pistas que puede presentar un editor de código o para generar documentación de forma automática, y están para ayudarle a saber con qué tipo de parámetros las funciones operan. Pero el intérprete Python **no las hace cumplir**. Al escribir tus propias funciones, depende de ti si quieres anotar tus tipos; pueden ser útiles para mantener organizados o no algo en lo que quieras pasar tiempo.

Q1 (3 puntos): Encontrar una golosina (dot) fija usando Búsqueda a Profundidad (DFS)

En searchAgents.py, encontrarás un completamente implementado SearchAgent, que planea un camino a través del mundo de Pacman y luego ejecuta ese camino paso a paso. Los algoritmos de búsqueda para formular un plan no se implementan su trabajo.

Primero, prueba que el SearchAgent está funcionando correctamente ejecutando:

```
$ python3 pacman.py -l tinyMaze -p SearchAgent -a fn=tinyMazeSearch
```

El comando de arriba dice a la SearchAgent de uso tinyMazeSearch como su algoritmo de búsqueda, que se implementa en search.py. Pacman debería navegar el laberinto con éxito.

Ahora es hora de escribir funciones de búsqueda genéricas completas para ayudar a Pacman plan rutas. Pseudo-codigo para los algoritmos de búsqueda que debe escribir, se puede encontrar en las diapositivas de la clase. Recuerde que un nodo de búsqueda debe contener no sólo un estado sino también la información necesaria para reconstruir el camino (plan) que llega a ese estado.

Nota importante: Todas sus funciones de búsqueda necesitan devolver una lista de acciones que llevarán al agente desde el principio a la meta. Todas estas acciones tienen que ser movimientos legales (direcciones válidas, no moverse a través de muros).

Nota importante: Asegúrese de usar las estructuras de datos Stack, Queue y PriorityQueue que se le han proporcionado en util.py. Estas implementaciones de la estructura de datos tienen propiedades particulares que se requieren para la compatibilidad con el auto-corrector.

Sugerencia: Cada algoritmo es muy similar. Los algoritmos para DFS, BFS, UCS y A* difieren sólo en los detalles de cómo se gestiona la frontera. Por lo tanto, concéntrate en conseguir DFS correcto y el resto debería ser relativamente sencillo. De hecho, una posible implementación requiere sólo un único método de búsqueda genérico que se configura con una estrategia de cola específica de algoritmo. (Su implementación no necesita ser de este formulario para recibir crédito completo).

Implementar el algoritmo de búsqueda de búsqueda a profundidad (DFS) en la función depthFirstSearch en search.py. Para completar su algoritmo, escriba la versión de búsqueda en la **versión de grafos** de DFS, que evita ampliar cualquier estado ya visitado.

Su código debe encontrar rápidamente una solución para:

```
$ python3 pacman.py -l tinyMaze -p SearchAgent
```

```
$ python3 pacman.py -l mediumMaze -p SearchAgent
```

```
$ python3 pacman.py -l bigMaze -z .5 -p SearchAgent
```

La pantalla de Pacman mostrará una superposición de los estados explorados, y el orden en que fueron explorados (rojo más brillante significa exploración anterior). ¿Es el orden de exploración lo que habrías esperado? ¿Pacman realmente va a todas las posiciones exploradas en su camino a la meta?

Sugerencia: Si usas la implementación dada Stack como su estructura de datos, la solución encontrada por su algoritmo DFS para mediumMaze debe tener una longitud de 130 pasos (siempre que introduzca los sucesores en la frontera en el orden proporcionado por getSuccessors; usted podría obtener 246 pasos si los empuja en el orden inverso). ¿Es esta una solución de menor costo? Si no, piense en qué profundidad está haciendo mal la búsqueda.

Puntuación: Por favor ejecute el siguiente comando para ver si su implementación pasa todos los casos de prueba de auto-corrector.

```
$python3 autograder.py -q q1
```

Q2 (3 puntos): Búsqueda a lo Ancho (BFS)

Implemente el algoritmo de primera búsqueda (BFS) en la función breadthFirstSearch en el archivo search.py. Una vez más, escriba un algoritmo de búsqueda basado en la versión de grafos que evite expandir cualquier estado ya visitado. Pon a prueba tu código de la misma manera que lo hiciste para la búsqueda de profundidad.

```
$ python3 pacman.py -l mediumMaze -p SearchAgent -a fn=bfs
```

```
$ python3 pacman.py -l bigMaze -p SearchAgent -a fn=bfs -z .5
```

BFS encuentra una solución de menor costo? Si no, comprueba tu implementación.

Sugerencia: Si Pacman se mueve demasiado lento para usted, intente la opción "frameTime 0".

Nota: Si ha escrito su código de búsqueda genéricamente, su código debe funcionar igualmente bien para el problema de búsqueda de ocho rompecabezas sin cambios.

```
$ python3 eightpuzzle.py
```

Puntuación: Por favor ejecute el siguiente comando para ver si su implementación pasa todos los casos de prueba de auto-corrector.

```
$ python3 autograder.py -q q2
```

Q3 (3 pts): Variación de la Función de Costos

Mientras que BFS encontrará un camino de menos acciones hacia la meta, tal vez queramos encontrar caminos que son "mejores" en otros sentidos. Considere mediumDottedMaze y mediumScaryMaze.

Mediante el cambio de la función de costos, podemos animar a Pacman a encontrar diferentes caminos. Por ejemplo, podemos penalizar (mayor coste) como pasos peligrosos en áreas plagadas de fantasmas o menos para pasos en áreas ricas en golosinas, y un agente Pacman racional debe ajustar su comportamiento en respuesta.

Implementar el algoritmo de búsqueda de gráficos de costo uniforme en la función uniformCostSearch en el archivo search.py. Te animamos a revisar util.py para algunas estructuras de datos que pueden ser útiles en su implementación. Ahora debe observar el comportamiento exitoso en los tres diseños siguientes, donde los agentes a continuación son todos agentes de UCS que difieren sólo en la función de costos que utilizan (los agentes y funciones de costo están escritos para usted):

```
$ python3 pacman.py -l mediumMaze -p SearchAgent -a fn=ucs
```

```
$ python3 pacman.py -l mediumDottedMaze -p StayEastSearchAgent
```

```
$ python3 pacman.py -l mediumScaryMaze -p StayWestSearchAgent
```

Nota: Usted debe obtener costos de ruta muy bajo y muy altos para el StayEastSearchAgent y StayWestSearchAgent respectivamente, debido a sus funciones de costo exponencial (véase searchAgents.py para los detalles).

Puntuación: Por favor ejecute el siguiente comando para ver si su implementación pasa todos los casos de prueba de auto-corrector.

```
$ python3 autograder.py -q q3
```

Q4 (3 puntos): Búsqueda A*

Implementar A* búsqueda en la función vacía aStarSearch en el archivo search.py. A* toma una función heurística como argumento. Las funciones que implementan heurísticas toman dos argumentos: un estado en el problema de búsqueda (el argumento principal), y el problema en sí (para información de referencia). La función heurística nullHeuristic en el archivo search.py es un ejemplo de una heurística trivial.

Usted puede probar su implementación de A* en el problema original de encontrar un camino a través de un laberinto hasta una posición fija usando la distancia de Manhattan como la función heurística. Esta heurística ya está implementada como una referencia en la función manhattanHeuristic en el archivo searchAgents.py.

Sugerencia: Además de hacer un seguimiento del estado visitado, también puede querer hacer un seguimiento de la ruta con el mejor costo de ruta hasta ese estado conseguido hasta este momento. Algunos nodos pueden necesitar expandirse más de una vez para encontrar el camino óptimo.

```
$ python3 pacman.py -l bigMaze -z .5 -p SearchAgent
     -a fn=astar,heuristic=manhattanHeuristic
```

Usted debe ver que A* encuentra la solución óptima ligeramente más rápido que la búsqueda de costos uniforme UCS (alrededor de 549 vs. 620 nodos de búsqueda expandidos en nuestra implementación, pero los lazos en prioridad pueden hacer que sus números difieran ligeramente). ¿Qué pasa en openMaze para las diversas estrategias de búsqueda?

Puntuación: Por favor ejecute el siguiente comando para ver si su implementación pasa todos los casos de prueba de auto-corrector.

```
$ python3 autograder.py -q q4
```

Q5 (3 puntos): Problema de Encontrar Todas las Esquinas

El poder real de A* sólo será evidente con un problema de búsqueda más desafiante. Ahora, es tiempo de formular un nuevo problema de búsqueda y diseñar unas heurísticas para él.

En los laberintos de esquina, hay cuatro puntos, uno en cada esquina. Nuestro nuevo problema de búsqueda es encontrar el camino más corto a través del laberinto que toque o visite cada una de las cuatro esquinas (si el laberinto realmente tiene comida allí o no es irrelevante). Tenga en cuenta que para algunos laberintos como tinyCorners, el camino más corto no siempre va a la comida más cercana primero. Sugerencia: el camino más corto a través tinyCorners da 28 pasos.

Nota: Asegúrese de completar la Pregunta 2 antes de trabajar en la Pregunta 5, porque la pregunta 5 se basa en su respuesta para la Pregunta 2.

Implementar el problema de búsqueda CornersProblem en el archivo searchAgents.py. Usted tendrá que elegir una representación del estado que codifique toda la información necesaria para detectar si se han alcanzado las cuatro esquinas. Ahora, su agente de búsqueda debería resolver:

```
$ python3 pacman.py -l tinyCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

```
$ python3 pacman.py -l mediumCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

Para recibir el crédito completo, es necesario definir una representación abstracta del estado que no codifique información irrelevante (como la posición de los fantasmas, donde hay golosinas extras, etc.). En particular, no utilice el GameState de Pacman como estado de búsqueda. Su código será muy, muy lento si lo hace (y también estaría incorrecto).

Una instancia de la clase que CornersProblem, representa todo un problema de búsqueda, no solamente un estado en particular. Los estados particulares son devueltos por las funciones que usted debe escribir

y proveer, y tus funciones devuelven una estructura de datos de tu elección (por ejemplo, tuple, conjunto, etc.) que representa un estado.

Además, mientras se ejecuta un programa, recuerde que muchos estados existen simultáneamente, todos en la cola del algoritmo de búsqueda, y deben ser independientes el uno del otro. En otras palabras, usted no debería tener sólo un estado para todo un objeto o instancia de CornersProblem. Su clase debería ser capaz de generar muchos estados diferentes para proporcionar al algoritmo de búsqueda.

Sugerencia 1*:* Las únicas partes del estado del juego que necesitas hacer referencia en tu implementación son la posición de inicio de Pacman y la ubicación de las cuatro esquinas.

Sugerencia 2: Al codificar getSuccessors, asegúrese de añadir hijos a su lista de sucesores con un costo de 1.

Nuestra implementación de breadthFirstSearch expande poco menos de 2000 nodos de búsqueda en mediumCorners. Sin embargo, los heurísticos (utilizados con búsqueda A*) pueden reducir la cantidad de búsqueda requerida.

Puntuación: Por favor ejecute el siguiente comando para ver si su implementación pasa todos los casos de prueba de auto-corrector.

```
$ python3 autograder.py -q q5
```

Q6 (3 puntos): Problema de Esquinas: Heurística

Nota: Asegúrese de completar la Pregunta 4 antes de trabajar en la Pregunta 6, porque la pregunta 6 se basa en su respuesta para la pregunta 4.

Implementar una heurística no trivial para el problema CornersProblem en la función cornersHeuristic.

```
$ python3 pacman.py -l mediumCorners -p AstarCornersAgent -z 0.5
```

Nota: AstarCornersAgent es un atajo para:

```
-p SearchAgent -a fn=aStarSearch,prob=CornersProblem,heuristic=cornersHeuristic
```

Admisibilidad: Recuerde, las heurísticas son sólo funciones que toman estados de búsqueda y devuelve números que estiman el costo a una meta más cercana. Una heurística más efectiva devolverá los valores más cercanos a los costos reales de los objetivos. Para ser *admisible*, los valores heurísticos deben ser **límites inferiores** en el camino más corto real al objetivo más cercano y no pueden tener un **valor negativo**.

Heurísticas No-Triviales: Las heurísticas triviales son las que devuelven cero en todos los estados (UCS) y la heurística que calcula el verdadero costo de finalización del algoritmo. El primero no te ahorrará tiempo ni esfuerzo, mientras que el segundo hará tiempo que el cronometro interno del auto-corrector expire. Usted quiere una heurística que reduzca el tiempo total de la computación, aunque para esta asignación el auto-corrector sólo comprobará los recuentos de nodos (aparte de hacer cumplir un plazo razonable).

Nota: Su heurística debe ser una heurística no-trivial como se definió anteriormente y no debe generar un valor negativo para **recibir cualquier clase de puntuación**. Asegúrese de que su heurística devuelve el valor 0 en cada estado objetivo o meta, y nunca devuelve un valor negativo. Dependiendo de los pocos nodos que se expande tu heurística, serás clasificado:

Número de Nodos Expandidos	Grado
Más de 2000	0/3
Hasta un máximo 2000	1/3
Como máximo 1600	2/3
Como máximo 1200	3/3

Puntuación: Por favor ejecute el siguiente comando para ver si su implementación pasa todos los casos de prueba de auto-corrector.

```
$ python3 autograder.py -q q6
```

Q7 (4 puntos): Comer Todos los Puntos

Ahora resolveremos un problema de búsqueda más difícil: comer todas las golosinas de Pacman en el menor número posible de pasos. Para ello, necesitaremos una nueva definición de **problema de búsqueda** que formalice el problema de la limpieza de todas las golosinas: FoodSearchProblem en searchAgents.py (implementado para que usted lo use).

Una solución se define como una ruta que recoge toda la comida en el mundo de Pacman. Para el proyecto actual, las soluciones no tienen en cuenta ningún fantasma o pellets de alimentación; las soluciones sólo dependen de la colocación de paredes, comida regular y Pacman. (Por supuesto que los fantasmas pueden arruinar la ejecución de una solución. Entraremos en el próximo proyecto.)

Si ha escrito correctamente sus métodos de búsqueda general, A* con una heurística nula (equivalente a la búsqueda de un costo uniforme UCS) debe encontrar rápidamente una solución óptima para testSearch sin ningún cambio de código por su parte (costo total de 7).

```
$ python3 pacman.py -l testSearch -p AstarFoodSearchAgent
```

Nota: AstarFoodSearchAgent es un atajo para:

```
-p SearchAgent -a fn=astar,prob=FoodSearchProblem,heuristic=foodHeuristic
```

Deberías encontrar que UCS empieza a ralentizarse, incluso para el aparentemente simple laberinto tinySearch. Como referencia, nuestra implementación toma 2,5 segundos para encontrar un camino de longitud 27 después de expandir 5057 nodos de búsqueda.

Nota: Asegúrese de completar la Pregunta 4 antes de trabajar en la Pregunta 7, porque la pregunta 7 se basa en su respuesta para la pregunta 4.

Complete el código de foodHeuristic en el archivo searchAgents.py con una heurística para el FoodSearchProblem. Prueba a tu agente en el laberinto trickySearch:

```
$ python3 pacman.py -l trickySearch -p AstarFoodSearchAgent
```

Nuestro agente de UCS encuentra la solución óptima en unos 13 segundos, expandiendo y explorando más de 16,000 nodos.

Cualquier heurística no-trivial y no-negativa debería obtener al menos 1 punto. Asegúrese de que su heurística devuelve 0 en cada estado objetivo y nunca devuelve un valor negativo. Dependiendo de los pocos nodos que se expanda tu heurística, obtendrás puntos adicionales:

Número de Nodos Expandidos	**Puntuacion**
Más de 15,000	1/4
Como máximo 15,000	2/4
Como máximo 12,000	3/4
Como máximo 9,000	4/4 (credito completo; dificultad mediana)
Como máximo 7,000	5/4 (crédito adicional opcional; dificultad alta)

Puntuación: Por favor ejecute el siguiente comando para ver si su implementación pasa todos los casos de prueba de auto-corrector.

```
$ python3 autograder.py -q q7
```

Q8 (3 puntos): Búsqueda Sub-Optima

A veces, incluso con A* y una buena heurística, encontrar el camino óptimo a través de todos los puntos es difícil. En estos casos, todavía nos gustaría encontrar un camino razonablemente bueno, de forma rápida. En esta sección, escribirás un agente que siempre se come de forma codiciosa (*greedy*) el punto más cercano. El agente, `ClosestDotSearchAgent`, se implementa para usted en `searchAgents.py`, pero falta una función clave que encuentra un camino al punto más cercano, y que usted debe completar.

Implementar la función `findPathToClosestDot` en el archivo `searchAgents.py`. Nuestro agente resuelve este laberinto de forma subóptima con un costo de 350, pero en un tiempo menor a un segundo:

```
$ python3 pacman.py -l bigSearch -p ClosestDotSearchAgent -z .5
```

Sugerencia: La forma más rápida de completar `findPathToClosestDot` es llenar un problema del tipo `AnyFoodSearchProblem`, al cual le está faltando su prueba de meta. A continuación, resolver ese problema con una función de búsqueda adecuada. La solución debe ser muy corta.

Tu agente `ClosestDotSearchAgent` no encontrará siempre el camino más corto posible a través del laberinto. Asegúrese de entender por qué y tratar de encontrar un pequeño ejemplo donde ir repetidamente al punto más cercano no resulta en encontrar el camino más corto para comer todos los puntos.

Puntuación: Por favor ejecute el siguiente comando para ver si su implementación pasa todos los casos de prueba de auto-corrector.

```
$ python3 autograder.py -q q7
```

Grado: Por favor ejecute el siguiente comando para ver si su implementación pasa todos los casos de prueba de autógrader.

python autograder.py -q q8

Entregables y Evaluación

Usted deberá entregar dos archivos por medio de la PVA:

1. Archivo **search.py** con su código editado.
2. Archivo **searchAgents.py** con su codigo editado.
3. Un archivo **README.txt** o **README.pdf** con explicaciones de como realizo su programa y que heurísticas creo.

Este último archivo ayudara más a la comprensión de su solución en caso de ser necesaria una revisión del mismo, pero no llevara peso en la evaluación de la asignación.

La evaluación suma **25 puntos**, los cuales serán convertidos a una **escala de 100** para determinar su valor real en su evaluación del curso.

Un resumen de cómo están distribuidos los puntos se da a continuación:

Pregunta	Puntuación
Q1	3
Q2	3
Q3	3
Q4	3
Q5	3
Q6	3
Q7	4
Q8	3
	25